Class Relationships

Lecture 18

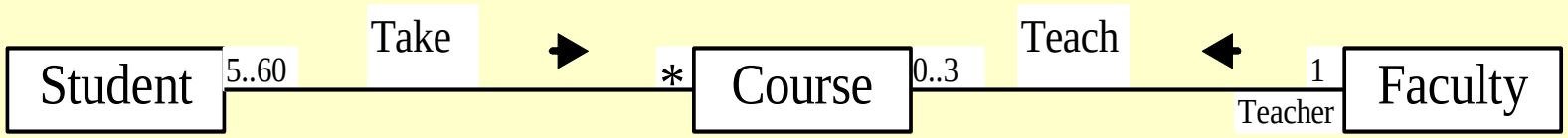
Based on Slides of Dr. Norazah Yusof

Relationships among Classes

- Association
- Aggregation
- Composition
- Inheritance

Association

Association **represents a general binary relationship that describes an activity between two classes.**




```
public class Student {
  /** Data fields */
  private Course[]
    courseList;

  /** Constructors */
  /** Methods */
}
```

```
public class Course {
  /** Data fields */
  private Student[]
    classList;
  private Faculty faculty

  /** Constructors */
  /** Methods */
}
```

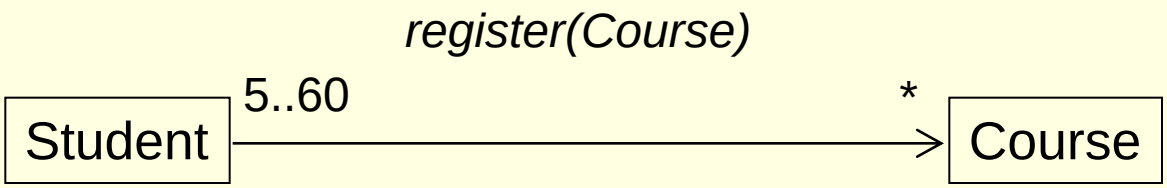
```
public class Faculty {
  /** Data fields */
  private Course[]
    courseList;

  /** Constructors */
  /** Methods */
}
```

An association is usually represented as a data field in the class.

Association

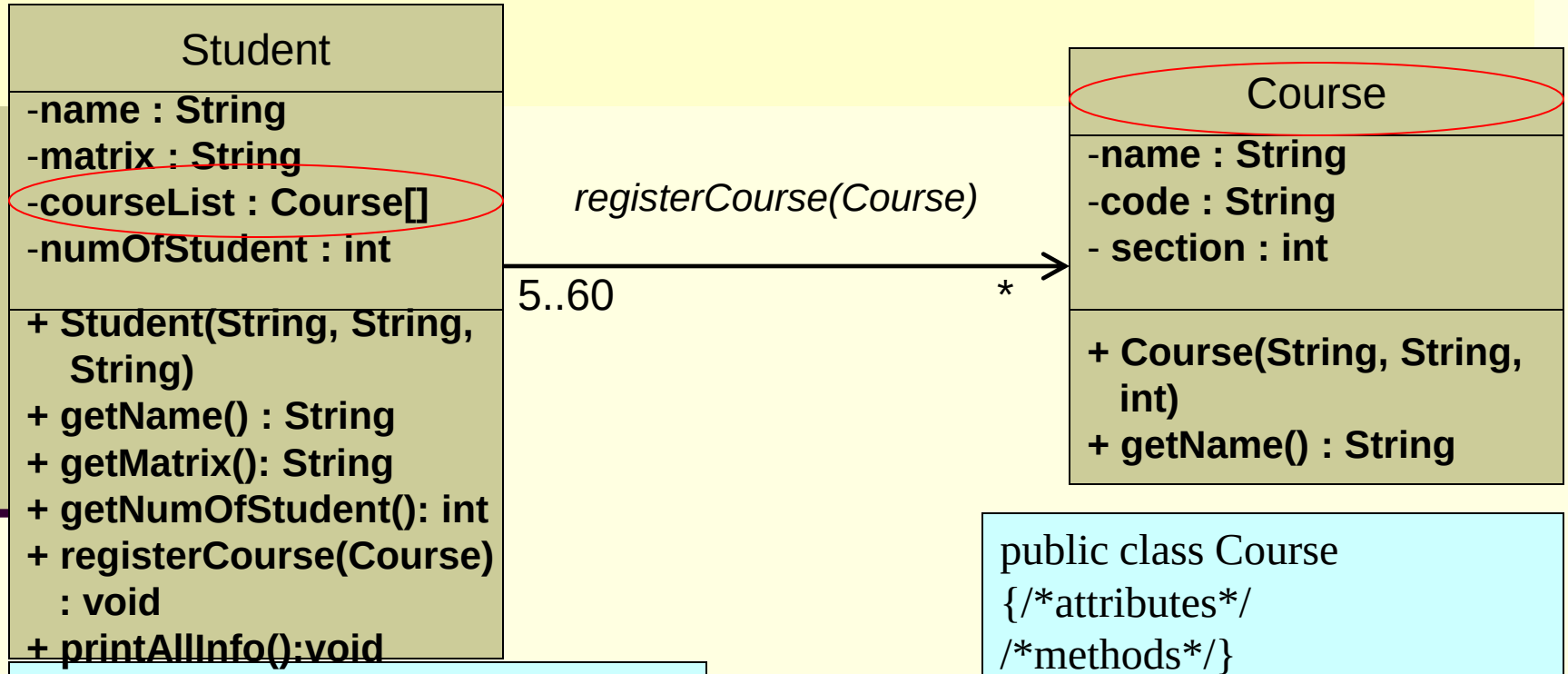
- **Example: Student register course.**



- **The arrow is optional and specifies navigability.**
- **No arrow implies bidirectional navigability**

Association

- **An association is usually represented as a data field in the class.**

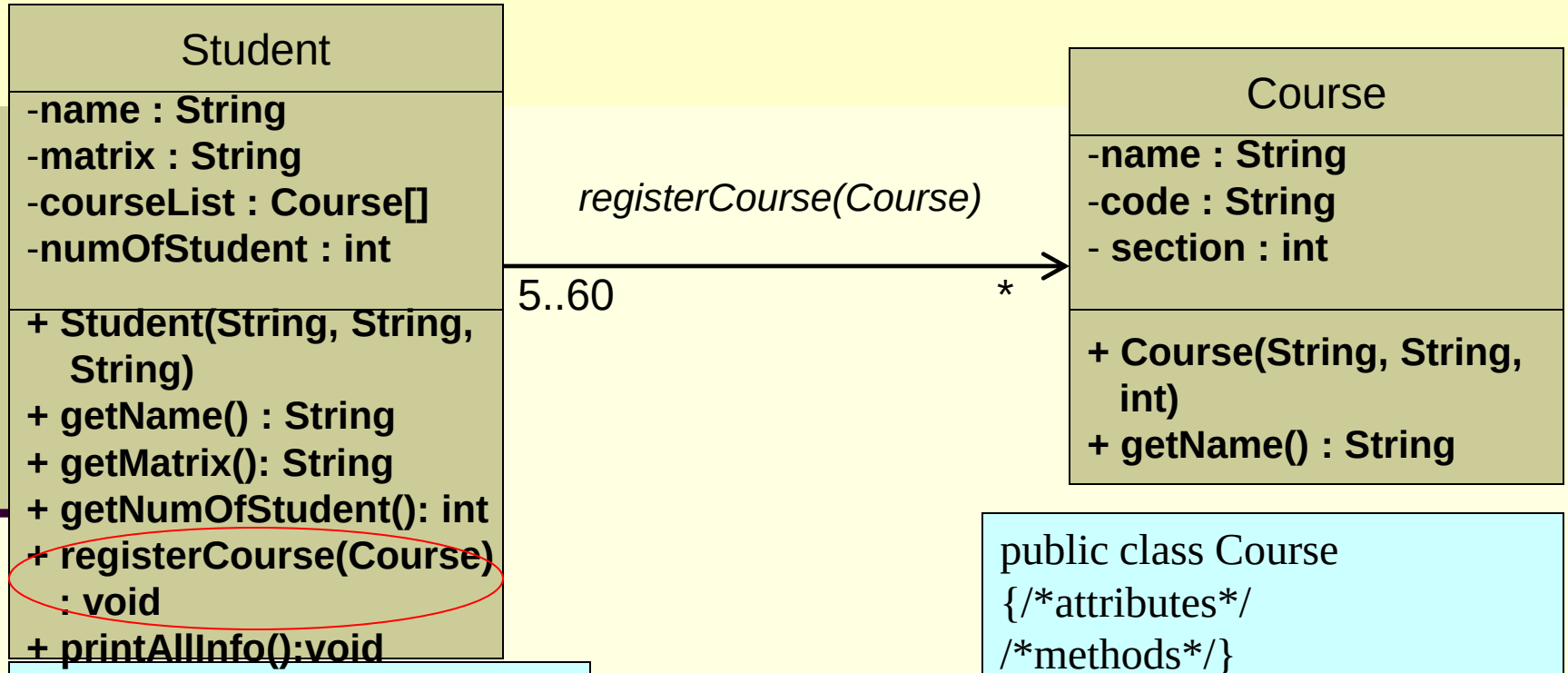



```
public class Course
{/*attributes*/
/*methods*/}
```

```
public class Student
{ /*attributes*/
  private Course[] courseList;
/*methods*/}
```

Association

- **The relationship allows objects to call methods in other objects.**

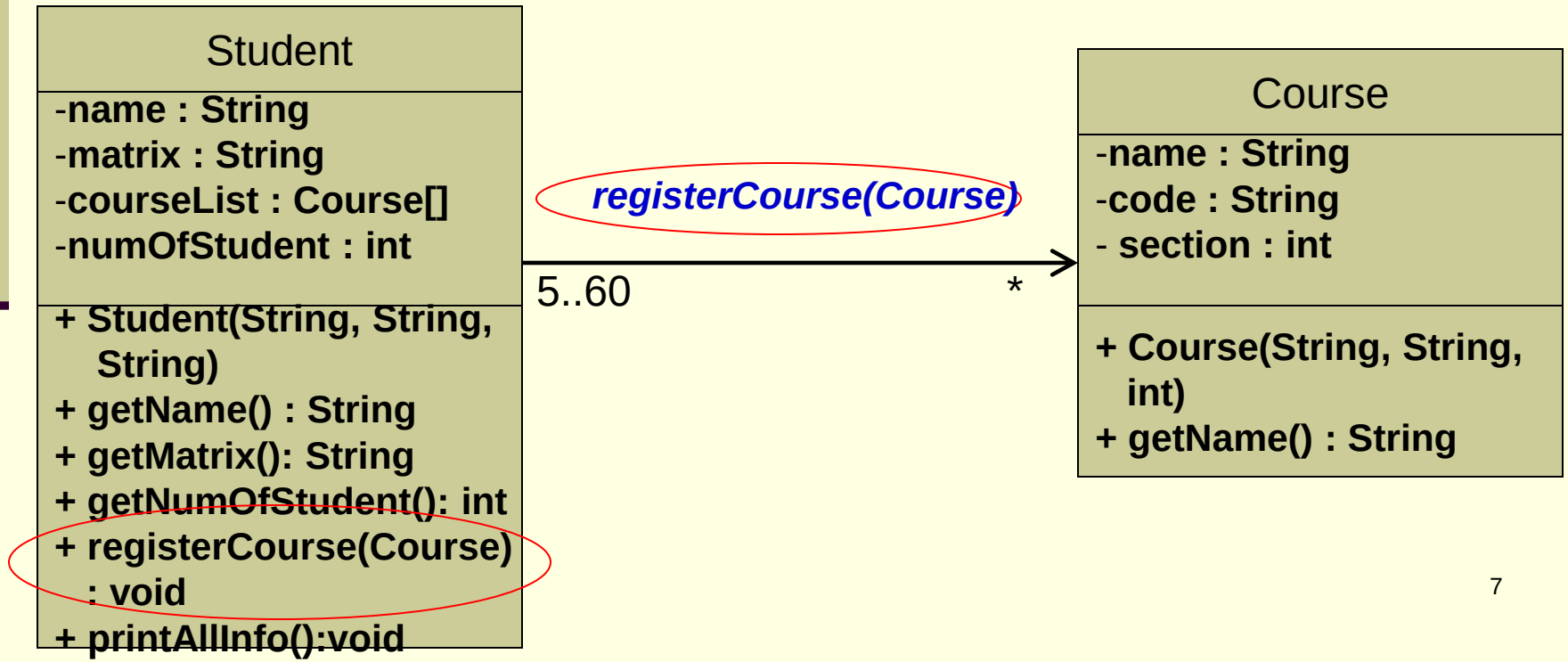



```
public class Course
{/*attributes*/
/*methods*/}
```

```
public class Student
{ /*attributes*/
  private Course[] courseList;
/*methods*/}
```

Association

- **This is the same as an object sending another object a message.**
- **Therefore, it is implemented as object references.**



Association Example 1: Student register Course

```
1  import java.util.*;
2  public class TestAssociate{
3    public static void main(String args[]){
4     Course cs1 = new Course("OOP","SCP3103",3);
5     Course cs2 = new Course("TP1","SCJ1013",3);
6     Course cs3 = new Course("TP2","SCJ1213",3);
7     Course cs4 = new Course("KP","SCP2113",3);
8
9    Student s1 = new Student ("ALI","AC1234","2SCS");
10    s1.registerCourse(cs1);
11    s1.registerCourse(cs2);
12    s1.printAllInfo();
13    System.out.println();
14
15   Student s2 = new Student ("AHMAD","AC1122","3SCK");
16    s2.registerCourse(cs1);
17    s2.registerCourse(cs3);
18    s2.registerCourse(cs4);
19    s2.printAllInfo();
20     }
21 }
```

Association Example 1: Student register Course (Student class)

```
public class Student {
  private String name;
  private String matrix;
  private Course[] courseList; //showing association
  private int numOfCourse;
  public Student(String n,String m,String c){
   name=n;
   matrix=m;
   courseList = new Course[10]; //showing association
  }
  public String getName() {
     return name;
  }
  public String getMatrix() {
     return matrix;
  }
```

Association Example 1: Student register Course (Student class – cont.)

```
public void registerCourse(Course cs)   //showing association
{   courseList[numOfCourse] = cs;
     numOfCourse++;
  }

  public int getNumOfCourse() {
     return numOfCourse;
  }
  public void printAllInfo() {
    System.out.println("\nSTUDENT NAME          :"+name);
    System.out.println("NUMBER OF SUBJECT(s) TAKEN :" +
       numOfCourse);
   System.out.println("LIST OF SUBJECT(s) TAKEN   :");
    for(int i=0;i<numOfCourse;i++) {
        Course s=(Course)courseList[i];
        System.out.println((i+1) + ". " + s.getName());
    }
  }
}
```

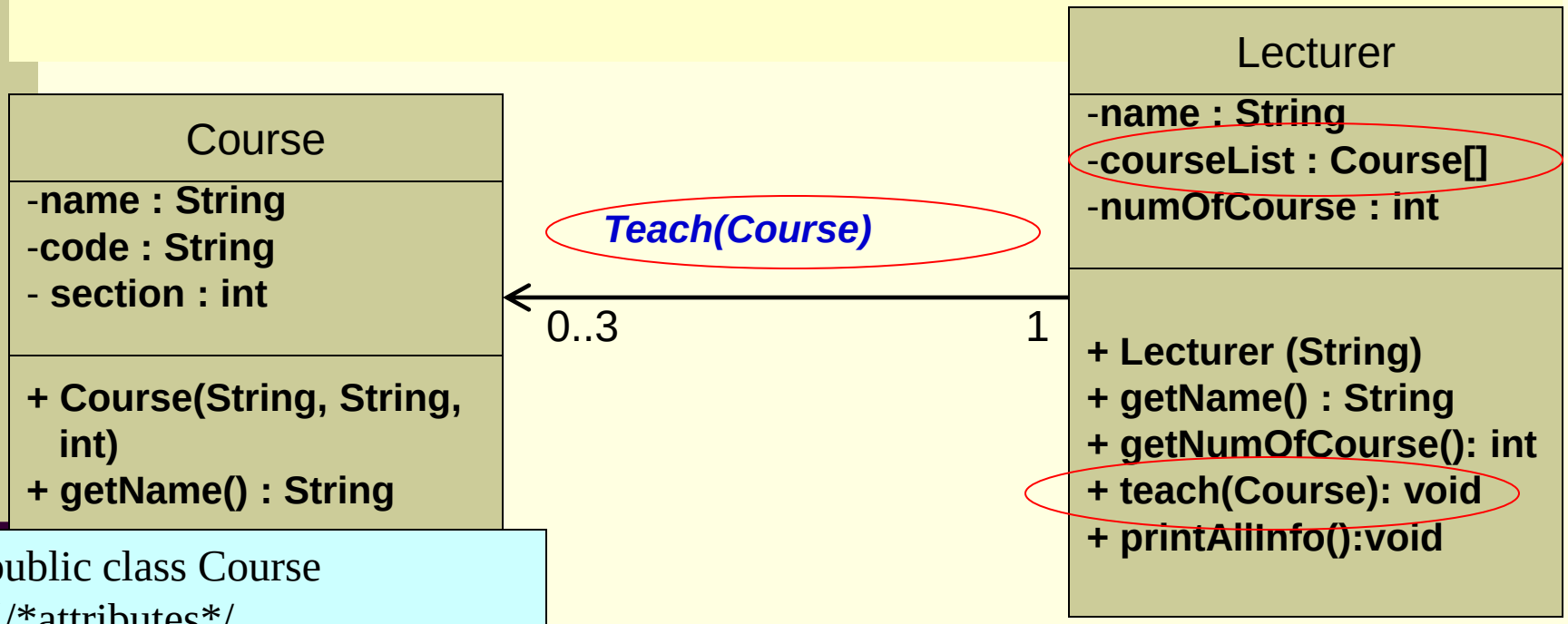
Association Example 1: Student register Course (Course class)

```
public class Course {
  private String name;
  private String code;
  private int section;
  public Course(String n,String c,int s){
     name = n;
    code = c;
    section =s;
  }

  public String getName() {
       return name;
  }
}
```

Association Example 2: Lecturer teach Course

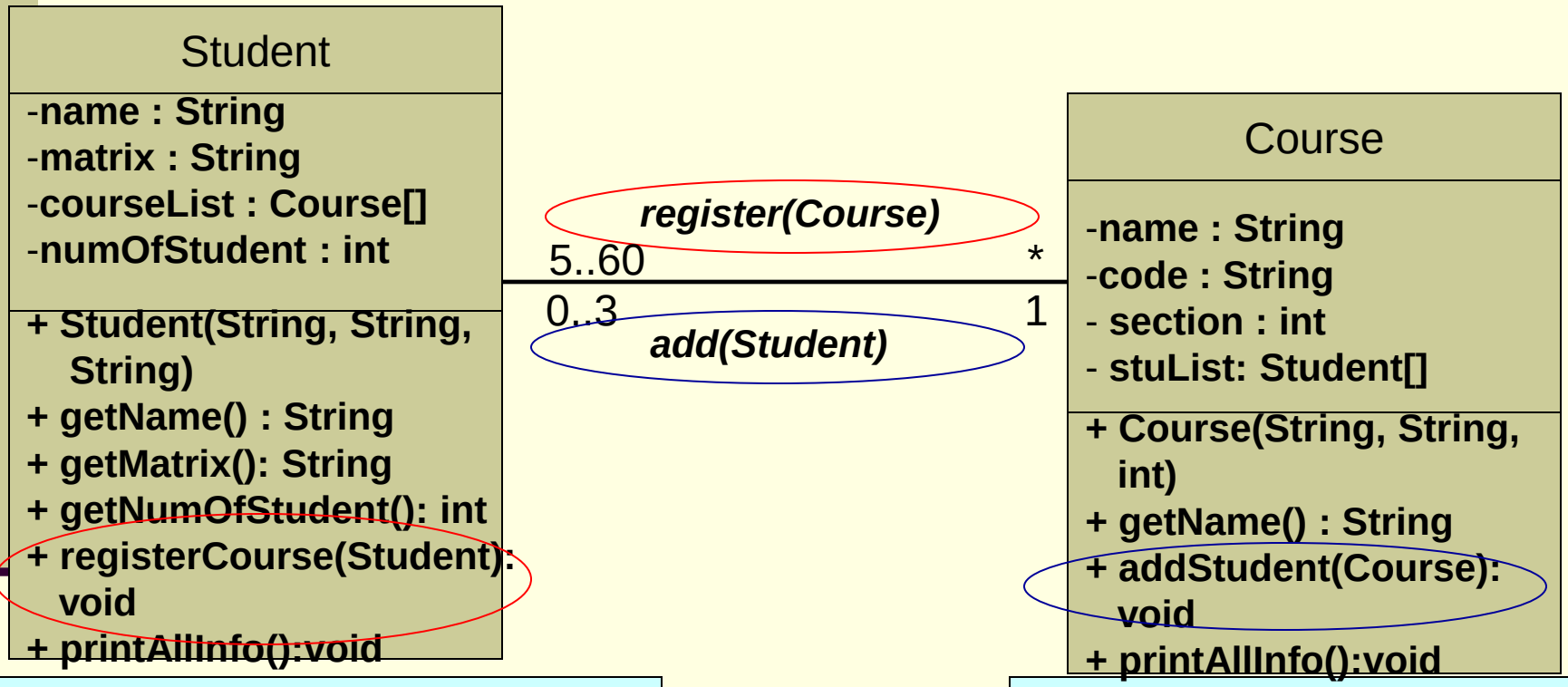
- **Write the classes and test the classes to show association relationships among them.**




```
public class Course
{/*attributes*/
/*methods*/}
```

```
public class Lecturer
{ /*attributes*/
  private Course[] courseList;
/*methods*/}
```

Association Example 3: Bidirectional Relationship
Student register Course and Course add Students




```
public class Student
{ /*attributes*/
  private Course[] courseList;
/*methods*/}
```

```
public class Course
{/*attributes*/
 private Student[] stuList;
/*methods*/}
```

Association Example 3: Bidirectional Relationship

Student register Course and Course add Students

```
import java.util.*;
public class TestAssociate2 {
  public static void main(String args[]){
   Course1 cs1 = new Course1("OOP","SCP3103",3);
   Course1 cs2 = new Course1("TP1","SCJ1013",3);
   Course1 cs3 = new Course1("TP2","SCJ1213",3);
   Course1 cs4 = new Course1("KP","SCP2113",3);
   Student1 s1 = new Student1("Ali","AC0021","2SCK");
     Student1 s2 = new Student1("Abu","AC0022","3SCK");
     Student1 s3 = new Student1("Ben","AC0023","3SCS");
     cs1.addStudent(s1);
     cs1.addStudent(s2);
     cs1.addStudent(s3);
     cs2.addStudent(s2);
     cs3.addStudent(s2);
     cs1.printAllInfo();
     cs2.printAllInfo();
     s1.printAllInfo();
     s2.printAllInfo();
     s3.printAllInfo();
    }
}
```

Association Example 3: Bidirectional Relationship
Student register Course and Course add Students

```
public class Student1 {
  private String name;
  private String matrix;
  private String major;
  private Course1[] courseList; //showing association
  private int numOfCourse;
  public Student1(String n,String m,String j){
   name=n;
   matrix=m;
   major = j;
   courseList = new Course1[4]; //showing association
  }
  public String getName() {
     return name;
  }
  public String getMatrix() {
     return matrix;
  }

```

Association Example 3: Bidirectional Relationship
Student register Course and Course add Students

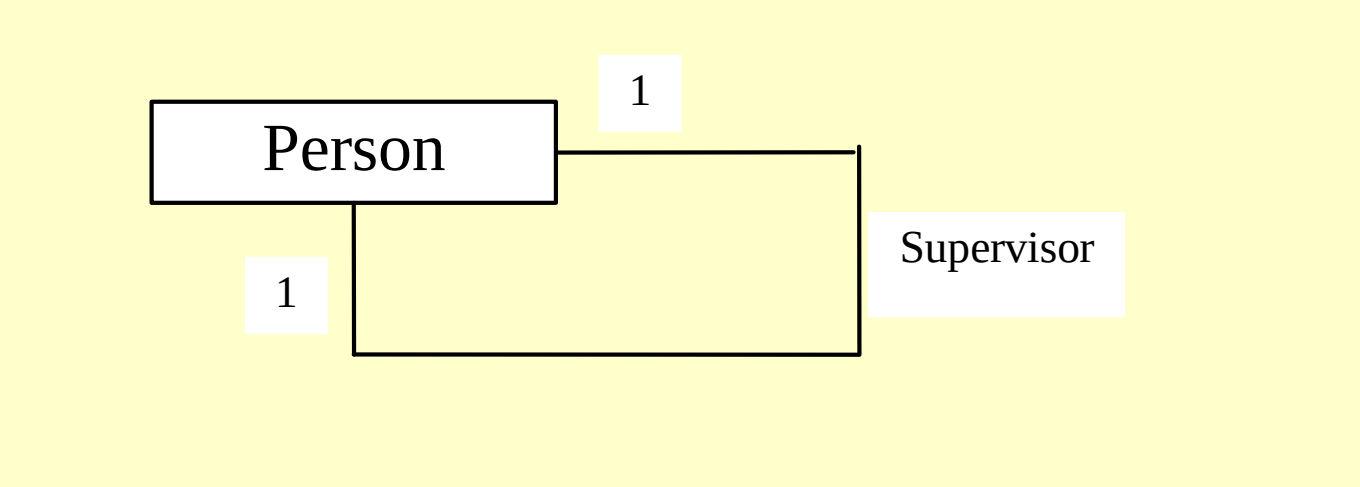
```
public String getMajor() {
      return major;
  }
  public void register(Course1 cs)  {      //showing association
     courseList[numOfCourse] = cs;
     numOfCourse++;
  }
  public int getNumOfCourse() {
     return numOfCourse;
  }
  public void printAllInfo() {
    System.out.println("\nSTUDENT  NAME            :"+name);
    System.out.println("NUMBER OF SUBJECT(s)
TAKEN :"+numOfCourse);
   System.out.println("LIST OF SUBJECT(s) TAKEN   :");
    for(int i=0;i<numOfCourse;i++) {
        Course1 s=(Course1)courseList[i];
        System.out.println((i+1) + ". " + s.getName());
    }
  }
}
```

Association Example 3: Bidirectional Relationship
Student register Course and Course add Students

```
public class Course1 {
  private String name;
  private String code;
  private int section;
  private Student1[] stuList; //showing association
  private int numOfStudent;
  public Course1(String n,String c,int s){
        name = n;
        code = c;
        section =s;
        stuList = new Student1[60];
   }
   public String getName()     {
        return name;
   }
     public void addStudent(Student1 st){    //showing association
      stuList[numOfStudent] = st;
      numOfStudent++;
      st.register(this);
   }//addStudent
```

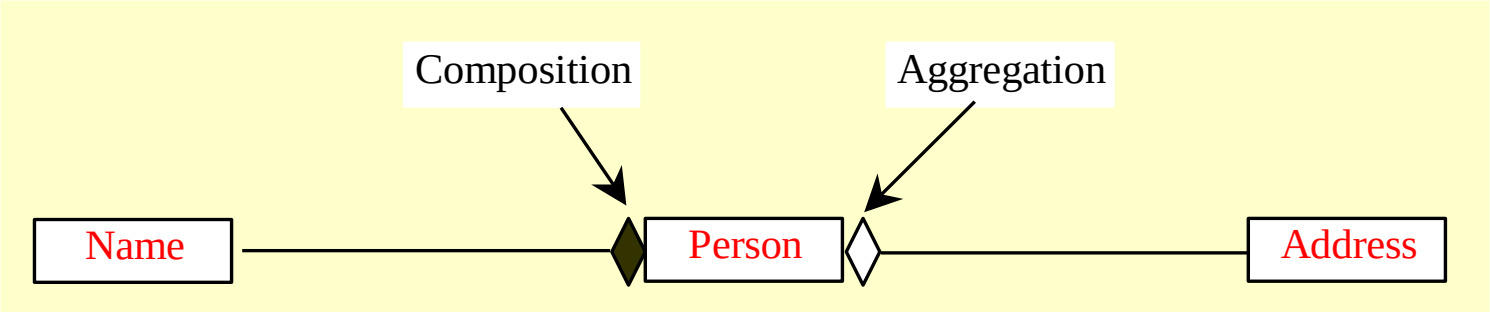
Association Between Same Class

Association may exist between objects of the same class. For example, a person may have a supervisor.



Aggregation and Composition

- ***Aggregation*** **is a special form of association, which represents an ownership relationship between two classes.**
- **Aggregation models the *has-a* relationship.**
- **If an object is exclusively owned by an aggregated object, the relationship between the object and its aggregated object is referred to as *composition*.**



Representing Aggregation in Classes

An aggregation relationship is usually represented as a data field in the aggregated class.

```
public class Name {
  /** Data fields */
  /** Constructors */
  /** Methods */
}
```

```
public class Person {
  /** Data fields */
  private Name name;
  private Address address;

  /** Constructors */
  /** Methods */
}
```

```
public class Address {
  /** Data fields */
  /** Constructors */
  /** Methods */
}
```

Inner Classes Translation

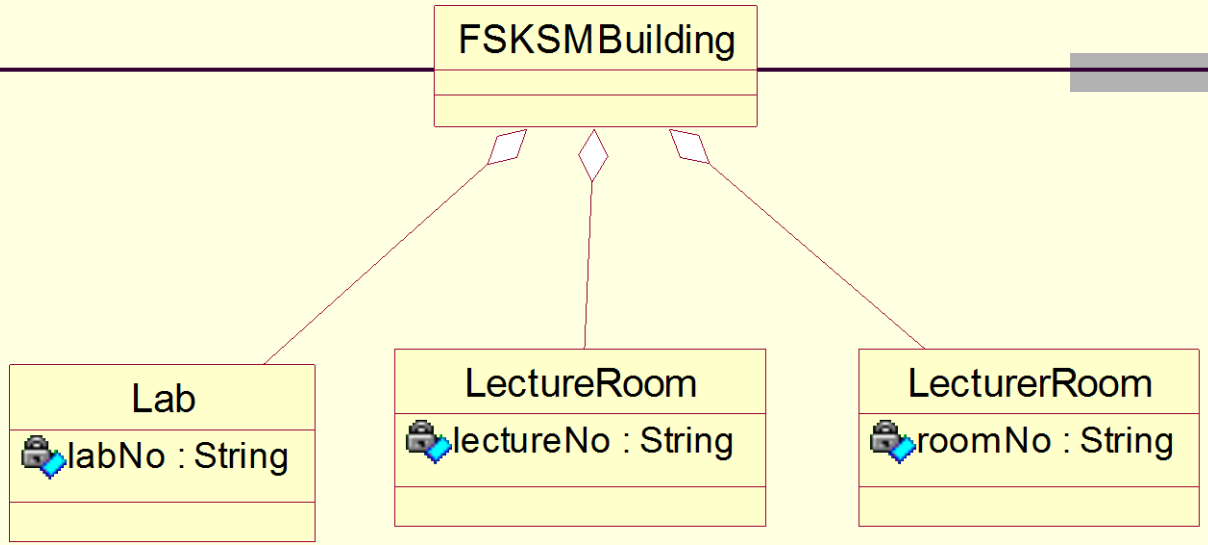
If Name or Address is used in the Person class only, they can be declared as an inner class in Person. For example,

```
public class Person {
  private Name name;
  private Address address;
  ...

  class Name {
  ...
  }

  class Address {
  ...
  }
}
```

Aggregation Example 1: FSKSM building has Lab, LectureRoom and LecturerRoom

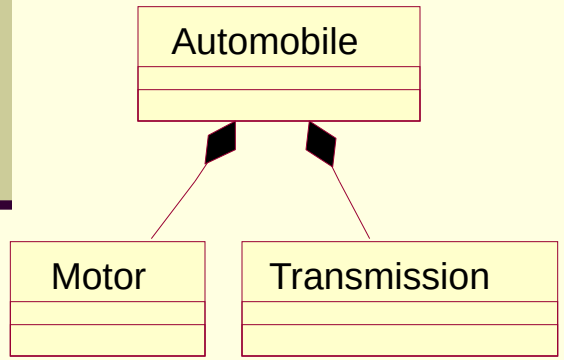



```
public class FSKSMBuilding
{       private Lab lab1;
        private LectureRoom lect;
        private LecturerRoom lectr;
}
```

Aggregation relationship is represented as a data field in the aggregated class.

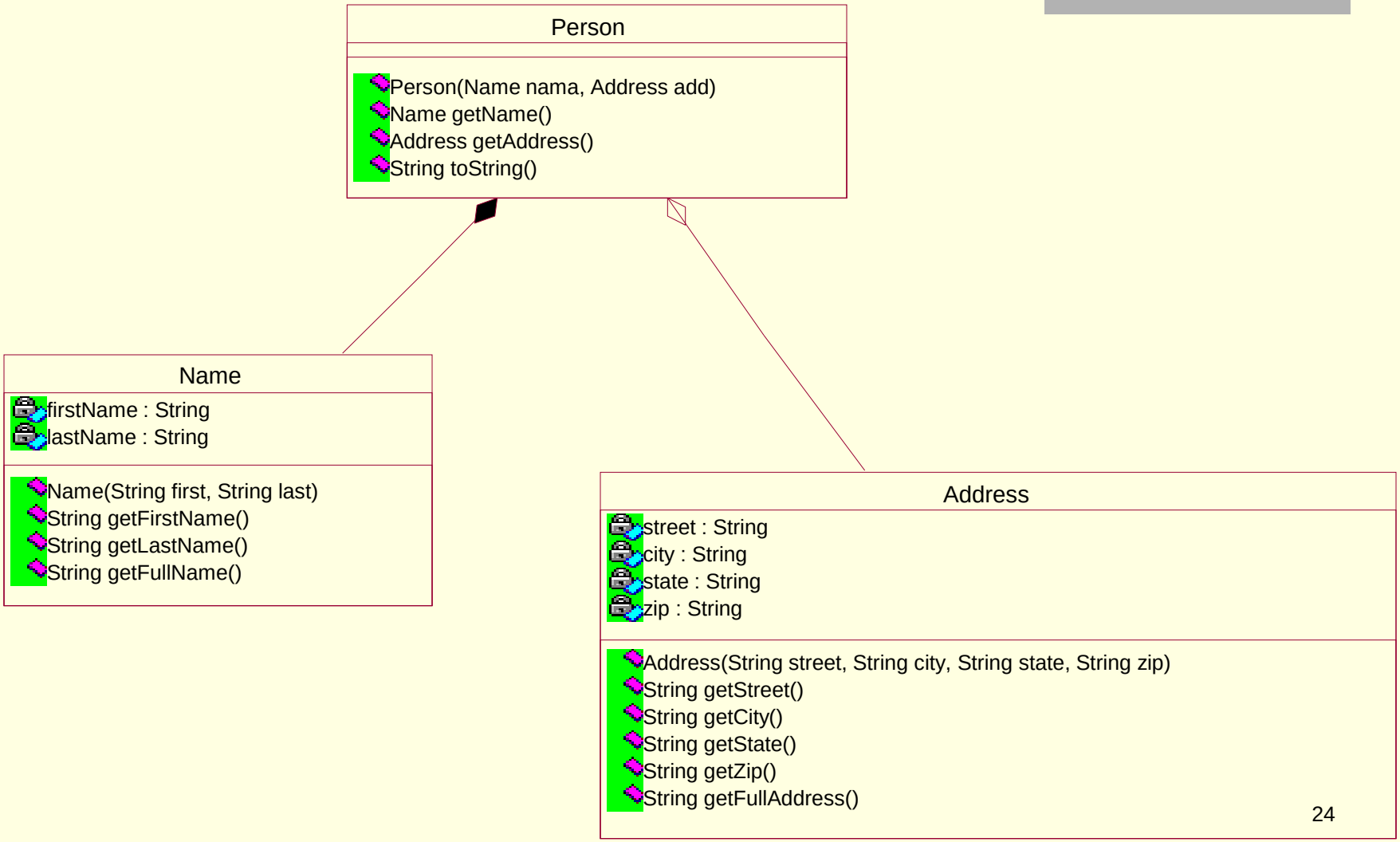
Composition

- Form of aggregation.
- For a relationship: "contains a".
- Strong ownership / binding.
- Parts belong to one whole.
- Parts do not change during execution.




```
public class Automobile
{
private final Transmission drive = new Transmission();
private final Motor engine = new Motor();
}
```

Composition Example 1 – A Person has a name and an address



Composition Example 1 – Name class

```
public class Name
{       private String firstName;
        private String lastName;
        public Name(String firstName, String lastName)
        {       this.firstName=firstName;
                this.lastName=lastName;
        }
        public String getFullName()
        {       return firstName + ' ' + lastName; } }
```

Composition Example 1 – Address class

```
public class Address
{       private String street;      private String city;
        private String state;       private String zip;
        public Address(String street, String city, String state, String zip)
        {       this.street = street;
                this.city = city;
                this.state = state;
                this.zip = zip;     }
                public String getFullAddress()
        {       return street +'\n' + city+ ", " +state+ ' ' +zip ; }
}
```

Composition Example 1 – Person has a name and an address

```
public class Person
{       private Name name;
        private Address address;

        public Person(Address1 address)
        {
                name1=new Name("Mohamad", "Ali");
                this.address=address;
        }
        public Name getName()
        {       return name;   }
        public String toString()
        {       return '\n' + name.getFullName() + '\n'
                +address.getFullAddress() +'\n'; }
}
```

Composition Example 1 – TestPerson

```
public class TestPerson {
  public static void main(String args[]){
    Address add1 = new Address("Jalan Pulai 13/2",
            "Skudai", "Johor", "81310");
    Person per1 = new Person(add1);
    System.out.println(per1);
  }
}
```

Self-Test

- **What is the appropriate relationship for the following classes:**
  - **Company and Employee**
  - **Faculty and Course**
  - **Customer and Product**
  - **Student and Matric Number**
  - **Employee and Address**
  - **House and Door**